L2 Observability & Monitoring / SRE

Questions & Spoken Answers

Prepared directly from the uploaded source document; terminology and answer framing are preserved.

1. Tell me how you would design observability for a new application from scratch.

Spoken answer:

Spoken answer: "I start by understanding the critical user journeys and defining the SLIs and SLOs. Then I instrument the application for metrics, logs and traces. For metrics, I would use Prometheus, Grafana for dashboards, ELK for centralized logs, and OpenTelemetry for distributed tracing. I would monitor the golden signals—latency, traffic, errors and saturation—and correlate metrics, logs and traces using common identifiers such as request or trace IDs. Then I define actionable alerts based on SLO impact, configure routing and escalation, create runbooks, and validate the complete flow with test incidents. The goal is not just collecting telemetry, but making it useful for detecting, diagnosing and resolving production issues."

PROMETHEUS

2. Explain Prometheus architecture.

Spoken answer: "Prometheus is primarily a pull-based monitoring system. Applications or exporters expose metrics through an HTTP endpoint, and Prometheus scrapes those endpoints at configured intervals. It stores the time-series data with labels, and I query it using PromQL. Alerting rules can evaluate PromQL expressions and send alerts through Alertmanager. In a Kubernetes environment, I commonly monitor nodes, pods, containers and application metrics using exporters and service discovery."

3. What are the four Prometheus metric types?

Spoken answer: "The four main metric types are Counter, Gauge, Histogram and Summary. Counter is monotonically increasing, such as total requests. Gauge can increase or decrease, such as CPU or memory usage. Histogram is useful for distributions such as request latency and allows bucket-based analysis. Summary also calculates quantiles on the client side. For production latency monitoring, I generally prefer histograms because they work well for aggregation across instances."

4. Counter vs Gauge?

Spoken answer: "A counter represents something that accumulates, such as requests, errors or bytes processed, and normally only increases until the process restarts. A gauge represents the current state and can move in both directions, such as CPU utilization, memory usage, queue depth or active connections."

5. Give me some practical PromQL queries.

Request rate

```
sum(rate(http_requests_total[5m]))
```

Error rate

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
```

CPU

```
100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

Memory

```
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
```

Pod restart count

```
increase(kube_pod_container_status_restarts_total[15m])
```

Say:

Spoken answer: "I use these queries as building blocks and then tune them based on the application's labels, service boundaries and SLOs."

GRAFANA

6. How would you design a production Grafana dashboard?

Spoken answer: "I avoid putting every available metric on one dashboard. I structure it around the service's health. At the top I show availability, error rate, request rate and latency percentiles. Then I provide infrastructure saturation such as CPU, memory, disk and network. Below that I show dependency health, database or messaging metrics, and deployment information. I also provide region, environment and service filters. The dashboard should help an on-call engineer answer three questions quickly: what is broken, who is impacted, and where should I investigate next."

7. P95 vs P99 latency?

Spoken answer: "P95 means 95% of requests completed at or below that latency and the slowest 5% are above it. P99 focuses on the slowest 1%. P95 is useful for general user experience, while P99 is useful for identifying tail latency problems. For customer-facing APIs, I normally monitor multiple percentiles rather than relying only on average latency."

8. Why is average latency dangerous?

Spoken answer: "Average can hide tail latency. For example, if most requests complete in 100 milliseconds but a small percentage take 10 seconds, the average might still look acceptable while some users experience severe delays. That's why I use percentiles such as P95 and P99 along with error rate and traffic."

ALERTING

9. How do you design an actionable alert?

Spoken answer: "I start with the symptom that requires human action rather than simply alerting on every metric crossing a threshold. The alert should have a clear condition, appropriate duration, severity, ownership, routing and runbook. I also consider whether the alert indicates customer impact or imminent SLO violation. I use grouping and deduplication to prevent multiple alerts for the same incident. After implementation, I review alert history and tune thresholds to reduce false positives without hiding genuine incidents."

10. You have 40 alerts per day and 75% are false positives. What do you do?

Spoken answer: "First I analyse the alert history and classify alerts into actionable, informational and noise. For noisy alerts, I investigate whether the threshold is too sensitive, the evaluation window is too short, or the alert is detecting a symptom that doesn't require human intervention. I then tune thresholds and durations, introduce grouping and deduplication, and route alerts according to severity. If something self-recovers without human action, it generally shouldn't page the on-call engineer. After tuning, I monitor the false-positive rate and make sure genuine incidents are still detected."

11. How do you decide whether CPU should alert at 80%, 85% or 90%?

Spoken answer: "I wouldn't choose the value arbitrarily. I establish the baseline from historical behaviour, workload patterns and capacity. More importantly, I correlate CPU utilization with customer-facing SLIs such as latency, error rate and throughput. If the service starts showing latency degradation around 80%, that could be an actionable threshold. If it remains healthy until 90%, then 80% may only be informational. So the threshold should be based on observed system behaviour and the consequences of saturation."

This is particularly important for your next round.

SLO / ERROR BUDGET

12. Explain SLI, SLO and SLA.

Spoken answer: "SLI is the actual measurement, such as availability or latency. SLO is the internal reliability target for that measurement, for example 99.9% availability. SLA is the external contractual commitment to the customer. I use SLOs to drive monitoring and alerting, and the error budget helps balance reliability work against feature delivery."

13. What is an error budget?

Spoken answer: "An error budget is the amount of unreliability permitted by the SLO. For example, with a 99.9% availability SLO, the remaining 0.1% represents the error budget. I use it as a decision-making mechanism. If we're consuming the budget too quickly, we should prioritise reliability and investigate the causes rather than continuing to introduce risky changes."

OBSERVABILITY

14. Monitoring vs observability?

Spoken answer: "Monitoring generally tells me that something is wrong based on predefined signals. Observability goes further by allowing me to understand why the system is behaving that way using metrics, logs and traces. For example, an alert might tell me that API latency increased, while tracing and logs can help me identify that the latency is coming from a particular database call or downstream service."

15. Explain the complete observability flow.

Spoken answer: "The application generates metrics, logs and traces. Metrics are collected by Prometheus, logs are forwarded into a central logging platform such as ELK, and traces can be generated using OpenTelemetry and sent to a tracing backend. Grafana or Kibana provides visualization and investigation. Alerting evaluates important conditions and routes actionable alerts to the on-call system such as PagerDuty or Opsgenie. The engineer then uses the dashboards, logs and traces to triage, mitigate and recover the incident, followed by RCA and preventive actions."

OPENTELEMETRY

16. What is OpenTelemetry?

Spoken answer: "OpenTelemetry is an open-source observability framework for generating and collecting telemetry, primarily metrics, logs and traces. I particularly use it from a tracing perspective to follow a request across multiple services. The important part is that the trace context is propagated between services, allowing me to correlate a user request with individual service calls and identify where latency or failures are occurring."

17. How would you troubleshoot a slow API using OpenTelemetry?

Spoken answer: "I start with the trace for a slow request and inspect the spans. I look at the total request duration and then identify which downstream span is consuming the most time. It could be a database query, another API, cache or messaging system. I correlate the trace ID with application logs and Prometheus metrics. Once I identify the bottleneck, I mitigate it—for example by scaling, fixing a dependency or rolling back a deployment—and then validate the latency against the SLO."

ELK

18. Explain ELK.

Spoken answer: "ELK stands for Elasticsearch, Logstash and Kibana. Logstash is commonly used for collecting and transforming logs, Elasticsearch stores and indexes the data, and Kibana provides search and visualization. In an incident, I would use Kibana to filter logs by service, environment, timestamp, severity and correlation or trace ID, then correlate those findings with metrics and traces."

Be honest if they ask about hands-on experience:

Spoken answer: "My strongest hands-on logging experience has been with Splunk and Azure Log Analytics rather than ELK. However, I'm familiar with the ELK architecture and log investigation workflow, and the underlying troubleshooting approach is similar."

> Do not claim ELK hands-on if you haven't done it.

APPLICATION TROUBLESHOOTING

19. User says checkout API is slow but all dashboards are green. What do you do?

Spoken answer: "I wouldn't conclude that the system is healthy just because infrastructure dashboards are green. I'd start with the user transaction and check the API's RED signals—request rate, error rate and latency. Then I'd inspect P95 and P99 latency and use OpenTelemetry traces to identify slow spans. I would check application logs, database latency, external dependencies, network or gateway latency and regional behaviour. I would also compare the current

deployment with the previous version. Once I isolate the bottleneck, I'd mitigate it and validate recovery using the same user-facing SLI."

20. API latency suddenly increases. Walk me through your investigation.

Spoken answer: "First I determine impact and scope—whether it's one service, region or all customers. Then I check traffic, error rate, P95/P99 latency and saturation. I correlate the timing with deployments or configuration changes. Next I inspect traces to identify slow dependencies and logs for errors or timeouts. I check database, cache, network, load balancer and downstream service health. If there's a recent bad deployment, I may roll it back. If it's capacity-related, I can scale after validating the cause. Once recovered, I monitor the SLO and then complete RCA and preventive actions."

INCIDENT MANAGEMENT

21. Explain your P1 incident process.

Spoken answer: "For a P1, I first acknowledge the incident and establish impact and scope. I bring in the required application, platform and dependency teams and establish an incident communication channel. I check metrics, logs and traces to identify the failing component. The immediate priority is mitigation and restoring service, so I may use rollback, failover, scaling or traffic management depending on the situation. After recovery I validate the SLIs, continue monitoring, document the timeline and conduct RCA with corrective and preventive actions."

22. What is the difference between mitigation and resolution?

Spoken answer: "Mitigation restores or stabilises the service without necessarily fixing the underlying cause. For example, scaling the service or rolling back a deployment can mitigate an incident. Resolution addresses the root cause. After mitigation, I investigate the underlying issue and implement a permanent corrective action through the appropriate change process."

23. What information should a runbook contain?

Spoken answer: "A good runbook should contain the alert definition, impact and symptoms, prerequisites, dashboards and logs to check, diagnostic commands, decision points, mitigation steps, rollback or recovery procedures, escalation contacts and validation steps. I also include links to relevant monitoring dashboards and dependencies. The objective is that another engineer can follow it during an incident without relying on tribal knowledge."

LINUX

24. How do you troubleshoot high CPU on Linux?

Spoken answer: "First I confirm the issue with top or uptime and identify which process is consuming CPU. I use ps or top to identify the process and inspect its command and PID. Then I check application logs and recent changes to understand why the process is consuming CPU. I also check memory, load average, disk I/O and whether the process is stuck or spawning excessive threads. If necessary, I mitigate by restarting or scaling the service according to the runbook, then investigate the root cause."

25. How do you find the top five CPU-consuming processes?

```
ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu | head -6

You can say:
```

Spoken answer: "I use `ps` sorted by CPU and take the top five processes."

26. How do you find large files?

```
du -ah /var | sort -rh | head -20

Or:

find /var -type f -size +1G -ls
```

27. How do you check disk usage?

```
df -h

For a specific directory:

du -sh /var/log/*
```

28. How do you investigate a process consuming excessive memory?

Spoken answer: "I identify the process using top or ps, then check its memory percentage and RSS. I inspect application logs and process behaviour, and determine whether memory usage is continuously increasing, which could indicate a leak. I also check whether the system or container is approaching its memory limit. In Kubernetes I would additionally check pod limits, OOMKilled events and container restart history."

KUBERNETES

29. Pod is restarting repeatedly. How do you troubleshoot?

Spoken answer: "I start with `kubectl get pods` and check the restart count and status. Then I use `kubectl describe pod` to inspect events and `kubectl logs`, including `--previous`, to see why the previous container terminated. I check whether it's an application crash, configuration problem, failed health probe, OOMKilled condition or dependency issue. I also check node resources and recent deployments. After identifying the cause, I apply the appropriate mitigation and verify that the pod becomes stable."

30. Pod is OOMKilled. What do you check?

Spoken answer: "I check the pod's memory requests and limits, actual usage and restart history. Then I determine whether the application genuinely requires more memory or whether there's abnormal growth or a memory leak. I inspect application metrics and logs and compare the behaviour with previous releases. Increasing the limit can be a mitigation, but I wouldn't treat that as the permanent fix without understanding why memory usage increased."

KAFKA / DISTRIBUTED SYSTEMS

31. Explain Kafka at a high level.

Spoken answer: "Kafka is a distributed event-streaming platform. Producers publish messages to topics, topics are divided into partitions, and brokers store those partitions. Consumers read messages and track their offsets. Consumer groups allow multiple consumers to process partitions in parallel. From an SRE perspective, I mainly monitor broker health, partition availability, consumer lag, throughput, errors and disk usage."

32. Consumer lag suddenly increases. What do you check?

Spoken answer: "First I confirm whether lag is isolated to one consumer group, topic or partition. Then I check consumer health, consumer processing rate and whether consumers are restarting or experiencing errors. I check partition distribution and broker health, including CPU, disk and network. I also investigate whether producer traffic suddenly increased or whether a downstream dependency is slowing consumer processing. The key is determining whether the problem is increased producer volume, insufficient consumer capacity, partition imbalance or downstream latency."

33. What happens if a Kafka consumer is down?

Spoken answer: "If it's part of a consumer group, Kafka can rebalance the partitions among the remaining consumers, depending on the group configuration and availability. But if there isn't enough consumer capacity, lag can increase. I would check consumer-group membership, partition assignment, lag, consumer errors and broker health, then restore or scale consumers as appropriate."

AWS

34. How do you monitor AWS applications?

Spoken answer: "I use CloudWatch for AWS-native metrics, logs and alarms, and depending on the application I correlate those with Prometheus and Grafana. For example, I might monitor EC2 CPU and memory through appropriate agents, ALB request count and latency, Lambda invocations and errors, and application logs through CloudWatch Logs. For incidents, I correlate infrastructure metrics with application-level SLIs rather than looking at infrastructure metrics in isolation."

35. How do you filter CloudWatch Logs?

```
For Logs Insights:

fields @timestamp, @message
```

```
| filter error = "critical"
| sort @timestamp desc
| limit 100
If the field is embedded in the message:
fields @timestamp, @message
| filter @message like /critical/
| sort @timestamp desc
```

DEVOPS / CI-CD

36. A deployment completed successfully but application health deteriorated. What do you do?

Spoken answer: "I compare the timing of the degradation with the deployment. I check application error rate, latency, logs and traces, and compare the new version with the previous version. If the deployment clearly correlates with the incident and rollback is safe, I would roll back to restore service. After recovery, I would investigate the change, identify the root cause and add appropriate tests, monitoring or deployment gates to prevent recurrence."

37. How do you make deployments safer?

Spoken answer: "I use automated CI/CD validation, infrastructure-as-code, code reviews, security and quality gates, health checks and progressive deployment strategies where appropriate. For production I prefer mechanisms such as canary or blue-green deployment when the platform supports them. I also monitor business and technical SLIs during deployment and maintain a straightforward rollback path."

ADVANCED L2 SCENARIOS

38. Everything looks normal but customers in one region are reporting failures. What do you do?

Spoken answer: "I immediately scope the issue by region and compare the affected region against healthy regions. I check regional traffic, latency, error rate, infrastructure health, DNS, load balancers, networking and dependency availability. Then I use logs and traces filtered by region to identify whether the problem is application-level or infrastructure-level. If the architecture allows it, traffic can potentially be shifted to a healthy region as mitigation. After recovery, I'd investigate why the regional failure wasn't detected earlier and improve the regional SLI or alerting."

39. Prometheus is showing no metrics from a service. How do you troubleshoot?

Spoken answer: "First I determine whether the problem is with the application endpoint, Prometheus scraping or service discovery. I check whether the `/metrics` endpoint is reachable and returning valid metrics. Then I check the Prometheus target status and scrape errors. In Kubernetes I check the ServiceMonitor or scrape configuration, labels, service and endpoints. I also check Prometheus logs and network policies. Once I identify whether it's instrumentation, discovery or connectivity, I fix that layer and validate that metrics are being scraped again."

40. Grafana shows "No Data". What do you check?

Spoken answer: "I first determine whether Prometheus itself has the data. Then I check the Grafana data-source health, time range, query and dashboard variables. If the query returns no data in Prometheus, I investigate the scrape or metric issue. If Prometheus has the data but Grafana doesn't, I investigate the data source, query or dashboard configuration."

41. Prometheus itself is down. What happens?

Spoken answer: "The immediate impact is loss of monitoring visibility and potentially alert evaluation, although the application may continue running. I would first determine whether it's a single Prometheus instance or the entire monitoring platform. I check its process or pod, logs, resource utilisation, storage and recent configuration changes. I restore Prometheus service as quickly as possible, then validate scraping and alert evaluation. For prevention, I'd look at HA architecture, storage capacity and monitoring Prometheus itself."

VERY LIKELY L2 QUESTION

42. How do you correlate metrics, logs and traces?

Spoken answer: "I use metrics to detect the symptom, logs to understand the detailed event, and traces to follow the request across services. The correlation becomes much more powerful when we propagate identifiers such as trace ID

and request ID into application logs. For example, if Prometheus shows P99 latency increasing, I can identify a slow trace, find the affected downstream service or database span, and then search the corresponding trace ID in the logs to understand the actual error or timeout."

43. What are RED and USE?

Spoken answer: "RED is mainly useful for request-driven services: Rate, Errors and Duration. USE is more infrastructure-oriented: Utilization, Saturation and Errors. I use RED for APIs and microservices and USE for infrastructure resources such as CPU, memory, disks and network. Together they provide a useful starting point for understanding both application and infrastructure health."

44. What are the four Golden Signals?

Spoken answer: "The four golden signals are latency, traffic, errors and saturation. Latency tells me how long requests take, traffic tells me demand, errors show failed requests, and saturation tells me how close the system is to its capacity limits. I use them as a starting point, but I also connect them to business SLIs and SLOs."

45. What would you do if an alert fires but there is no customer impact?

Spoken answer: "I wouldn't immediately dismiss it. I'd determine whether it's an early warning, an infrastructure symptom or simply a noisy alert. I check the SLO impact and historical behaviour. If it's a useful predictive signal, I may keep it but change the severity or routing. If it consistently requires no action, I'd tune or remove it through the appropriate change process."

46. What makes an alert bad?

Spoken answer: "An alert is bad when it fires frequently without requiring human action, has no clear owner, lacks context, or doesn't represent meaningful customer or system impact. A good alert tells the engineer what is wrong, how severe it is, where it is happening, and what action or runbook should be followed."

47. How would you monitor a payment service?

Spoken answer: "I'd monitor the complete payment journey rather than just infrastructure. The core SLIs would include successful payment rate, request rate, P95/P99 latency and failure rate. I'd monitor dependencies such as the payment gateway, database, cache and messaging system. Metrics would go to Prometheus, dashboards to Grafana, logs to the central logging platform and distributed traces through OpenTelemetry. Alerts would be based on customer impact and SLOs, with severity-based routing and a runbook for common payment failures."

48. What if payment success rate suddenly drops from 99.9% to 95%?

Spoken answer: "I'd first confirm the drop isn't a monitoring problem. Then I'd establish the scope—region, API, payment method, version and dependency. I'd check error codes and logs and use traces to identify where transactions are failing. I'd compare the timing with recent deployments and dependency health. If a recent deployment is responsible, I'd roll back if appropriate. If a third-party gateway is failing, I'd follow the established failover or escalation process. Once recovered, I'd validate the success-rate SLI and investigate the root cause."

49. How would you reduce MTTR?

Spoken answer: "I focus on earlier detection, better telemetry and faster diagnosis. That means meaningful SLIs and SLOs, actionable alerts, good dashboards, correlated logs and traces, clear ownership, tested runbooks and automated remediation where appropriate. During incidents, clear roles and communication prevent duplicated effort. After incidents, I use RCA and post-incident actions to eliminate recurring failure modes."

50. What would you improve in your current observability setup?

```
This is a very good L2 question for you.
```

Spoken answer: "I'd look at it from three areas: coverage, signal quality and response. First, I'd ensure critical user journeys have proper metrics, logs and traces. Second, I'd review alert quality—particularly false positives, duplicate alerts and missing alerts. Third, I'd improve correlation between Grafana, logs and traces so engineers can move from detection to root cause quickly. I'd also review SLO-based alerting and runbook coverage. The objective is to reduce both

MTTD and MTTR without creating additional alert fatigue."

Production Troubleshooting Framework

Impact → Triage → Metrics/Logs/Traces → Isolate → Mitigate → Recover → RCA → Prevent

Use this framework for P1/P2 incidents, latency, alerting and distributed-system questions. Start with impact and scope, use telemetry to isolate the failing component, mitigate safely, validate recovery against the SLI/SLO, then complete RCA and prevention.